



Policy RAG Agent

Architecture Document — Hybrid Retrieval Design (Current)

The complete technical architecture of Agent 3 in the Prior Authorization pipeline: chunked policy knowledge base, Azure OpenAI embedding retrieval with keyword hybrid scoring, deterministic criteria evaluation, and graceful offline degradation.

System	Prior Authorization AI — multi-agent workflow
Component	Agent 3 · Policy RAG (<code>agents/policy_rag.py</code>)
Retrieval mode	Hybrid — chunk embeddings + keyword/fuzzy, automatic fallback
Embedding model	Azure OpenAI <code>text-embedding-3-small</code> (1536 dims)
Knowledge base	32 insurer policies → ~150 semantic chunks
Verified	90/90 labeled cases · 51/51 policy accuracy · zero regressions
Document date	July 2026

Where the agent sits, and what it owns

The Policy RAG agent is the third of five agents. It receives de-identified, schema-validated clinical facts and answers exactly one question: **“Which insurer policy governs this request, and does the documented evidence satisfy that policy’s criteria?”**

```
Extractor → ICD Coder → POLICY RAG → Form Filler → Critique
      ▲
      |
      | ExtractedClinicalFacts (Pydantic-typed, PHI already masked)
      |
      ▼
      |
      | PolicyCheckResult (policy_id · match · matched/missing criteria
      |                   · confidence · explainable rationale)
```

Two design invariants define the whole architecture:

Invariant 1 — Probabilistic retrieval, deterministic judgment

Embeddings and fuzzy matching may influence *which policy document is fetched* (where tolerance to phrasing helps). Whether that policy’s rules are met is decided by fixed rule logic only (where fuzziness would be unsafe). No LLM ever approves or rejects anything.

Invariant 2 — Degrade, never fail

Every external dependency (Azure credentials, network, cache file) has a per-call fallback to the fully offline keyword path. A dead API can slow one case; it can never error one.

The two layers inside the agent



3 · COMPONENT INVENTORY

Every moving part, file by file

COMPONENT	LOCATION	RESPONSIBILITY
Agent shell	agents/policy_rag.py	PHI boundary re-check · orchestrates retrieve → evaluate · blends confidence · emits typed <code>Handoff[PolicyCheckResult]</code>
Retriever + chunker	rag/retriever.py	<code>_build_chunks()</code> (policy → chunks) · <code>_semantic_scores()</code> (cosine per chunk, max per policy) · hybrid <code>retrieve()</code> · deterministic <code>evaluate_criteria()</code>
Embedding client	rag/embeddings.py	Azure OpenAI embeddings call · sha256-keyed persistent cache · cosine helper · api-version retry · fail-open to keyword
Policy knowledge base	dataset/legacy_full/insurer_policies/policies.json	32 structured policies: <code>procedure_keywords</code> , <code>diagnosis_keywords</code> , criteria checklist
Vector store (lightweight)	logs/embedding_cache.json	Persisted vectors keyed by (deployment, sha256 of text) — corpus embedded once; identical queries never re-billed; makes hybrid scoring reproducible run-to-run
Configuration	config.py + .env	<code>AZURE_OPENAI_*</code> credentials · <code>AZURE_OPENAI_EMBEDDING_DEPLOYMENT_NAME</code> · kill-switch <code>PRIOR_AUTH_USE_EMBEDDING_RETRIEVAL=false</code>
Output contract	schemas/policy_check.py	<code>PolicyCheckResult</code> — Pydantic-validated; downstream agents and the Critique verifier consume it as-is

4 · RUNTIME SEQUENCE

What happens on every case, in order

#	STEP	DETAIL	COST
1	PHI guard	<code>contains_phi()</code> over diagnosis narrative + procedure; ERROR handoff (workflow halt) on any hit	free
2	Build query	full diagnosis narrative + requested procedure (narrative keeps late-sentence disease names in scope)	free
3	Keyword scores	token overlap + fuzzy containment vs. every policy's keyword sets	free
4	Semantic scores	embed query (1 API call, then cached) · corpus chunks already cached · cosine vs. ~150 chunks · best chunk per policy	~1 embedding call/case, \$0 on repeats
5	Hybrid ranking	$0.55 \cdot kw + 0.45 \cdot sem$ per policy; top-1 wins; winning chunk recorded	free
6	Criteria evaluation	three deterministic check types against the facts; produces matched / missing lists	free
7	Confidence + handoff	$0.3 \cdot retrieval + 0.7 \cdot criteria_ratio$; Pydantic-validated result; rationale quotes mode, score and winning chunk	free

Scoring formulas (all deterministic given cached vectors)

$retrieval_score = 0.55 \times keyword_score + 0.45 \times rescale(best_chunk_cosine, 0.15 \rightarrow 0.70)$

$agent_confidence = 0.3 \times retrieval_score + 0.7 \times (criteria_met \div criteria_total)$

Keyword keeps the majority retrieval share deliberately: it is exact, word-traceable evidence. The semantic term is the tie-breaker that absorbs paraphrase ("knee arthroplasty" $\approx$ "knee replacement", "unresponsive to non-operative measures" $\approx$ "failed conservative therapy").

5 · CHUNKING SCHEME

How a policy becomes searchable chunks

Chunk granularity mirrors the document's own structure, so a semantic hit is always attributable to a specific part of the policy:

```
// POL-ORTHO-TKR-001 "Total Knee Replacement Medical Necessity Policy" becomes:  
chunk 1 "...Policy. Procedure covered: total knee replacement, knee arthroplasty, TKR..."  
chunk 2 "...Policy. Applicable diagnosis: osteoarthritis, degenerative joint disease..."  
chunk 3 "...Policy. Requirement: Severe osteoarthritis documented"  
chunk 4 "...Policy. Requirement: Conservative therapy failed for at least 12 weeks"  
chunk 5 "...Policy. Requirement: Imaging confirms joint degeneration"
```

- **Policy score** = max cosine over its chunks → one strongly-matching requirement is enough to surface a policy.
- **Explainability** — the winning chunk is quoted verbatim in the output rationale, e.g. *"best-matching chunk: 'TAVR Policy. Requirement: Echocardiogram confirms severe aortic stenosis'"*.
- The title is prefixed to every chunk so short requirement texts still carry their parent-policy context into the embedding.

Degradation behaviour

CONDITION	BEHAVIOUR	CASE IMPACT
No embedding deployment configured	Semantic path never attempted; pure keyword retrieval	None — offline mode
<code>PRIOR_AUTH_USE_EMBEDDING_RETRIEVAL=false</code>	Semantic path disabled by operator	None
Wrong/typo'd <code>AZURE_OPENAI_API_VERSION</code>	One automatic retry on GA version <code>2024-02-01</code> ; working client kept	None
API error / network down mid-run	That call returns keyword-only; client disabled for the process (no per-case timeout tax)	Retrieval mode noted as “keyword” in rationale
Cache file corrupt / unwritable	Rebuilt in memory; persistence skipped silently	Slightly more API calls; zero errors
PHI detected in input	ERROR handoff → workflow suspends as PHI violation	Case halted <i>by design</i> , before any external call

Trust properties

- **PHI:** masked upstream by the Extractor; re-verified at this agent's entry; only masked text is ever sent to the embedding API.
- **Secrets:** credentials live in `.env` (git-ignored); code reads them via `config.py` only.
- **Reproducibility:** cached vectors make hybrid scores identical run-to-run; criteria verdicts are rule-based; the audit trace records score, mode and rationale per case.
- **Independent verification:** Agent 5 (Critique) re-checks that the form's `policy_id` matches this agent's selection and that the decision is consistent with the criteria outcome.

Measured results (90 labeled cases)

METRIC	KEYWORD-ONLY	HYBRID (CURRENT)
Policy selection accuracy	51 / 51 checked	51 / 51 checked
Workflow status changes vs. baseline	—	0 of 90
Paraphrase robustness (“arthroplasty” test)	correct, low margin	correct, higher score & margin
Cost per repeated case	\$0	\$0 (cache hit)

Bottom line

The hybrid upgrade added semantic robustness for future, less-templated notes at zero measured regression on the current corpus — while preserving the offline path, deterministic criteria gates, and full auditability the workflow was built on.